МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил:

Д. А. Марковский

студент группы ПО-13

Проверил:

А. А. Крощенко

Доцент кафедры ИИТ

20.03.2026

Брест 2026

Цель работы:

Научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API.

Задание:

5) Построение карты взаимодействий разработчика на GitHub.
Напишите Python-скрипт, который:
1. Запрашивает у пользователя имя пользователя GitHub.
2. Использует API GitHub для получения списка всех репозиториев, в которых этот пользователь:
• Делал коммиты
• Открывал pull requests
• Создавал issues
• Ставил звёзды
3. Для каждого репозитория находит других разработчиков, с которыми пользователь взаимодействовал (например, авторы коммитов, ревьюеры pull requests, авторы issues).
4. Создаёт граф связей между пользователем и другими разработчиками.
5. Визуализирует граф с помощью networkx и matplotlib:
• Узлы (nodes) — пользователи
• Рёбра (edges) — взаимодействия (коммиты, PR, issues, звёзды)
6. Сохраняет граф в формате PNG/JPEG.

Код программы:

```
import os
import time
import json
from collections import defaultdict
import requests
from dotenv import load_dotenv
import networkx as nx
import matplotlib.pyplot as plt

load_dotenv()


class GitHubAPIClient:
    def __init__(self):
        self.base_url = "https://api.github.com"
        self.token = os.getenv('GITHUB_TOKEN')
        self.headers = {
            'Accept': 'application/vnd.github.v3+json',
            'Authorization': f'token {self.token}'
        }

    def make_request(self, url, params=None):
        try:
            response = requests.get(url, headers=self.headers, params=params,
timeout=60)
            if response.status_code == 403:
                if 'rate limit' in response.text.lower():
                    reset_time = int(response.headers.get('X-RateLimit-Reset',
0))
                    wait_time = max(reset_time - time.time(), 0)
                    print(f"Превышен лимит запросов. Ожидание {wait_time:.0f}
секунд...")
                    time.sleep(wait_time + 1)
                    return self.make_request(url, params)
```

```
                print("Доступ запрещен (403)")
                return None
            response.raise_for_status()
            return response.json()
        except requests.exceptions.RequestException as e:
            print(f"Ошибка при запросе: {e}")
            return None

    def get_all_pages(self, url, params=None):
        results = []
        page = 1

        if params is None:
            params = {}

        while True:
            params['page'] = page
            params['per_page'] = 100
            data = self.make_request(url, params.copy())
            if data is None:
                break
            if isinstance(data, list):
                if len(data) == 0:
                    break
                results.extend(data)
                if len(data) < 100:
                    break
            else:
                break
            page += 1
        return results


class CollaborationCollector:
    INTERACTION_TYPES = {
        'commit_authors': 'Авторы коммитов',
        'pr_reviewers': 'Ревьюеры PR',
        'pr_authors': 'Авторы PR (в чужих репозиториях)',
        'issue_authors': 'Авторы issues',
        'issue_commenters': 'Комментаторы issues',
        'starred_owners': 'Владельцы репозиториев со звездами'
    }

    def __init__(self, username):
        self.username = username.lower()
        self.collaborators = {
            'commit_authors': set(),
            'pr_reviewers': set(),
            'pr_authors': set(),
            'issue_authors': set(),
            'issue_commenters': set(),
            'starred_owners': set()
        }
        self.edge_details = defaultdict(lambda: defaultdict(int))

    def _add_collaborator(self, collaborator_type, username):
        if username and username.lower() != self.username:
            username_lower = username.lower()
            self.collaborators[collaborator_type].add(username_lower)
            self.edge_details[username_lower][collaborator_type] += 1

    def add_pr_author(self, author):
        self._add_collaborator('pr_authors', author)

    def add_issue_author(self, author):
        self._add_collaborator('issue_authors', author)
```

```
    def add_starred_owner(self, owner):
        self._add_collaborator('starred_owners', owner)

    def add_multiple_collaborators(self, collaborator_type, usernames):
        for username in usernames:
            self._add_collaborator(collaborator_type, username)

    def get_all_collaborators(self):
        all_collaborators = set()
        for collab_set in self.collaborators.values():
            all_collaborators.update(collab_set)
        return all_collaborators

    def get_edge_weight(self, username):
        return sum(self.edge_details[username].values())

    def get_interaction_types(self, username):
        return dict(self.edge_details[username])

    def get_collaborators_list(self):
        return sorted(self.get_all_collaborators())

    def to_dict(self):
        return {
            'username': self.username,
            'collaborators': list(self.get_all_collaborators()),
            'interaction_details': {
                user: dict(interactions)
                for user, interactions in self.edge_details.items()
            }
        }


class InteractionCollector:
    def __init__(self, username, api_client, collaboration_collector):
        self.username = username
        self.api = api_client
        self.collaboration = collaboration_collector
        self.commit_repos = set()
        self.pr_repos = set()
        self.issue_repos = set()
        self.starred_repos = set()

    @staticmethod
    def _extract_usernames_from_items(items, key='user'):
        usernames = set()
        for item in items:
            if key in item and item[key] and 'login' in item[key]:
                usernames.add(item[key]['login'])
        return usernames

    def get_user_repos(self):
        print("\nПолучаем репозитории пользователя...")
        url = f"{self.api.base_url}/users/{self.username}/repos"
        repos = self.api.get_all_pages(url)
        print(f"Найдено {len(repos)} репозиториев")
        return repos

    def check_commits_in_repo(self, owner, repo):
        url = f"{self.api.base_url}/repos/{owner}/{repo}/commits"

        params = {'author': self.username, 'per_page': 100}
        user_commits = self.api.get_all_pages(url, params)
        has_user_commits = bool(user_commits and len(user_commits) > 0)

        params = {'per_page': 100}
        all_commits = self.api.get_all_pages(url, params)
```

```
        if all_commits:
            authors = self._extract_usernames_from_items(all_commits, 'author')
            self.collaboration.add_multiple_collaborators('commit_authors',
authors)

        return has_user_commits

    def check_prs_in_repo(self, owner, repo):
        url = f"{self.api.base_url}/repos/{owner}/{repo}/pulls"
        params = {'state': 'all', 'per_page': 100}
        pulls = self.api.get_all_pages(url, params)
        found_user_prs = False
        if pulls:
            for pr in pulls:
                pr_author = pr['user']['login']
                if pr_author.lower() == self.username.lower():
                    found_user_prs = True
                    reviews_url = pr['url'] + '/reviews'
                    reviews = self.api.get_all_pages(reviews_url)
                    if reviews:
                        reviewers = self._extract_usernames_from_items(reviews,
'user')

self.collaboration.add_multiple_collaborators('pr_reviewers', reviewers)
                elif owner == self.username:
                    self.collaboration.add_pr_author(pr_author)
                comments_url = pr['comments_url']
                comments = self.api.get_all_pages(comments_url)
                if comments:
                    commenters = self._extract_usernames_from_items(comments,
'user')

self.collaboration.add_multiple_collaborators('issue_commenters', commenters)

        return found_user_prs

    def check_issues_in_repo(self, owner, repo):
        url = f"{self.api.base_url}/repos/{owner}/{repo}/issues"
        params = {'state': 'all', 'per_page': 100}
        issues = self.api.get_all_pages(url, params)
        found_user_issues = False
        if issues:
            for issue in issues:
                issue_author = issue['user']['login']
                if issue_author.lower() == self.username.lower():
                    found_user_issues = True
                    comments_url = issue['comments_url']
                    comments = self.api.get_all_pages(comments_url)
                    if comments:
                        commenters = self._extract_usernames_from_items(comments,
'user')

self.collaboration.add_multiple_collaborators('issue_commenters',
                                                                    commenters)
                elif owner == self.username:
                    self.collaboration.add_issue_author(issue_author)
                    comments_url = issue['comments_url']
                    comments = self.api.get_all_pages(comments_url)
                    if comments:
                        commenters = self._extract_usernames_from_items(comments,
                                                                      'user')

self.collaboration.add_multiple_collaborators('issue_commenters',
                                                                    commenters)

        return found_user_issues
```

```
    def get_starred_repositories(self):
        url = f"{self.api.base_url}/users/{self.username}/starred"
        starred_data = self.api.get_all_pages(url)

        if starred_data:
            for repo in starred_data:
                repo_full_name = repo['full_name']
                self.starred_repos.add(repo_full_name)
                owner = repo_full_name.split('/')[0]
                self.collaboration.add_starred_owner(owner)
        return self.starred_repos

    def collect_interaction_repos(self):
        repos = self.get_user_repos()
        if not repos:
            return

        for repo in repos:
            repo_name = repo['full_name']
            owner, repo_name_only = repo_name.split('/')
            if self.check_commits_in_repo(owner, repo_name_only):
                self.commit_repos.add(repo_name)
            if self.check_prs_in_repo(owner, repo_name_only):
                self.pr_repos.add(repo_name)
            if self.check_issues_in_repo(owner, repo_name_only):
                self.issue_repos.add(repo_name)
        self.get_starred_repositories()

    def get_repos_list(self):
        all_repos = self.commit_repos | self.pr_repos | self.issue_repos |
self.starred_repos
        return sorted(all_repos)


class GraphVisualizer:
    EDGE_COLORS = {
        'commit_authors': '#2ecc71',
        'pr_reviewers': '#3498db',
        'pr_authors': '#9b59b6',
        'issue_authors': '#e74c3c',
        'issue_commenters': '#f39c12',
        'starred_owners': '#1abc9c'
    }

    def __init__(self, username, collaboration_collector):
        self.username = username
        self.collaboration = collaboration_collector
        self.graph = nx.Graph()

    def build_graph(self):
        self.graph.add_node(self.username, type='central', weight=10)
        all_collaborators = self.collaboration.get_all_collaborators()
        for collaborator in all_collaborators:
            weight = self.collaboration.get_edge_weight(collaborator)
            self.graph.add_node(collaborator, type='collaborator', weight=weight)
            interactions = self.collaboration.get_interaction_types(collaborator)
            for interaction_type, count in interactions.items():
                self.graph.add_edge(
                    self.username,
                    collaborator,
                    type=interaction_type,
                    weight=count,
                    color=self.EDGE_COLORS.get(interaction_type, '#95a5a6')
                )

        return self.graph
```

```
    def visualize(self, output_filename='github_graph.png', figsize=(15, 10)):
        if self.graph.number_of_nodes() <= 1:
            return

        plt.figure(figsize=figsize)

        pos = nx.spring_layout(self.graph, k=2, iterations=50)

        other_nodes = [n for n in self.graph.nodes() if n != self.username]

        nx.draw_networkx_nodes(
            self.graph, pos,
            nodelist=[self.username],
            node_color='#e74c3c',
            node_size=3000,
            node_shape='o'
        )

        if other_nodes:
            node_sizes = [300 + 100 * self.graph.nodes[n].get('weight', 1) for n
in other_nodes]
            nx.draw_networkx_nodes(
                self.graph, pos,
                nodelist=other_nodes,
                node_color='#3498db',
                node_size=node_sizes,
                node_shape='o',
                alpha=0.8
            )

        for edge in self.graph.edges(data=True):
            if len(edge) == 3:
                u, v, data = edge
                color = data.get('color', '#95a5a6')
                weight = data.get('weight', 1)

                nx.draw_networkx_edges(
                    self.graph, pos,
                    edgelist=[(u, v)],
                    width=weight * 2,
                    edge_color=color,
                    alpha=0.6,
                    style='solid'
                )

        nx.draw_networkx_labels(
            self.graph, pos,
            {node: node for node in self.graph.nodes()},
            font_size=10,
            font_weight='bold'
        )

        legend_elements = []
        for interaction_type, color in self.EDGE_COLORS.items():
            display_name = CollaborationCollector.INTERACTION_TYPES.get(
                interaction_type,
                interaction_type.replace('_', ' ').title()
            )
            legend_elements.append(plt.Line2D(
                [0], [0],
                color=color,
                lw=4,
                label=display_name,
                alpha=0.6
            ))

        plt.legend(handles=legend_elements, loc='upper left',
```

```
                    bbox_to_anchor=(1, 1))
        plt.title(f"Граф взаимодействий пользователя {self.username}",
                  fontsize=16, fontweight='bold')
        plt.axis('off')
        plt.tight_layout()
        plt.savefig(output_filename, dpi=300, bbox_inches='tight')
        print(f"Визуализация графа сохранена в: {output_filename}")
        plt.show()


def save_to_json(data, filename='github_network.json'):
    with open(filename, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False, indent=2)
    print("Граф сохранён в ", filename)


def main():
    username = input("Введите имя пользователя GitHub: ").strip()
    if not username:
        print("Имя пользователя не может быть пустым!")
        return

    api_client = GitHubAPIClient()
    collaboration_collector = CollaborationCollector(username)
    collector = InteractionCollector(
        username=username,
        api_client=api_client,
        collaboration_collector=collaboration_collector
    )
    collector.collect_interaction_repos()
    repos_list = collector.get_repos_list()
    collaborators_list = collaboration_collector.get_collaborators_list()
    print(f"\nРепозитории ({len(repos_list)}):")
    for repo in repos_list:
        print(f"  - {repo}")
    print(f"\nКонтрибьюторы ({len(collaborators_list)}):")
    for collab in collaborators_list:
        print(f"  - {collab}")
    visualizer = GraphVisualizer(username, collaboration_collector)
    visualizer.build_graph()
    json_data = {
        'username': username,
        'repositories': repos_list,
        'collaborators': collaboration_collector.to_dict()
    }
    save_to_json(json_data, 'github_network.json')
    visualizer.visualize(output_filename='github_network.png')


if __name__ == "__main__":
    main()
```

Спецификация ввода

Введите имя пользователя GitHub: <имя пользователя>

Пример

Введите имя пользователя GitHub: carrotist-coder

Спецификация вывода

Получаем репозитории пользователя…

Репозитории (<число репозиториев>):

<список репозиториев>

Контрибьюторы (<число контрибьюторов>):

<список контрибьюторов>

Граф сохранён в github_network.json

Визуализация графа сохранена в github_network.png

Пример

Получаем репозитории пользователя…

Репозитории (2):

- carrotist-coder/BeatSalers

- carrotist-coder/Finances-Accounting

Контрибьюторы (2):

- aorgish

- demns

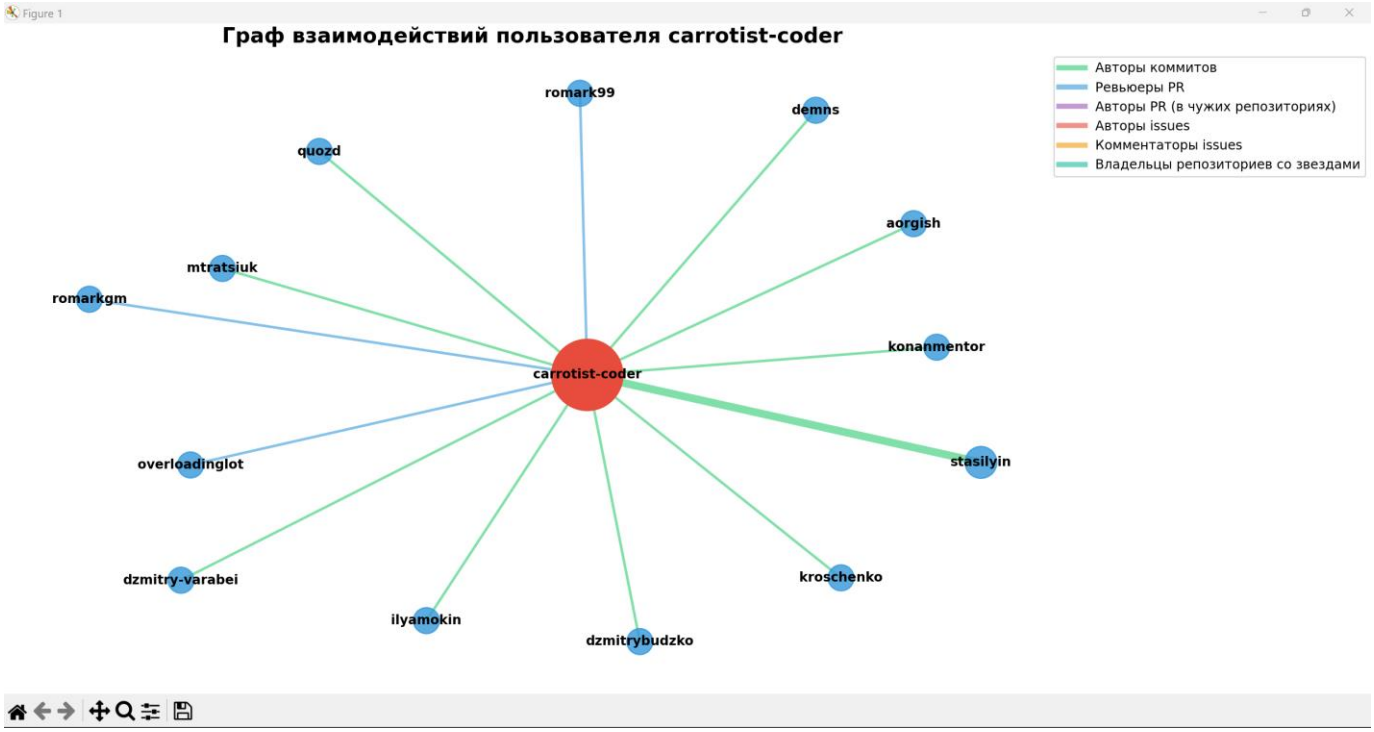
Граф сохранён в github_network.json

Визуализация графа сохранена в github_network.png

Рисунки с результатами работы программы

```
Введите имя пользователя GitHub: carrotist-coder

Получаем репозитории пользователя...
Найдено 11 репозиториев

Репозитории (11):
  - carrotist-coder/BeatSalers
  - carrotist-coder/Finances-Accounting
  - carrotist-coder/Matrix
  - carrotist-coder/Melodician-Sound-Libraries-Service
  - carrotist-coder/Rubik-s-Cube
  - carrotist-coder/if-html
  - carrotist-coder/if-js
  - carrotist-coder/if-react
  - carrotist-coder/js-assignments
  - carrotist-coder/pomr_po13
  - carrotist-coder/spp_po13

Контрибьюторы (13):
  - aorgish
  - demns
  - dzmitry-varabei
  - dzmitrybudzko
  - ilyamokin
  - konanmentor
  - kroschenko
```

```
  - carrotist-coder/Rubik-s-Cube
  - carrotist-coder/if-html
  - carrotist-coder/if-js
  - carrotist-coder/if-react
  - carrotist-coder/js-assignments
  - carrotist-coder/pomr_po13
  - carrotist-coder/spp_po13

Контрибьюторы (13):
  - aorgish
  - demns
  - dzmitry-varabei
  - dzmitrybudzko
  - ilyamokin
  - konanmentor
  - kroschenko
  - mtratsiuk
  - overloadinglot
  - quozd
  - romark99
  - romarkgm
  - stasilyin
Граф сохранён в  github_network.json
Визуализация графа сохранена в: github_network.png

Process finished with exit code 0
```



Вывод:

Я научился работать с Github API, приобрел практические навыки написания программ для работы с REST API.